

The following pages are taken from Algorithms Illuminated by Tim Roughgarden.

3.2 Counting Inversions in $O(n \log n)$ Time

3.2.1 The Problem

This section studies the problem of computing the number of inversions in an array. An *inversion* of an array is a pair of elements that are “out of order,” meaning that the element that occurs earlier in the array is bigger than the one that occurs later.

Problem: Counting Inversions

Input: An array A of distinct integers.

Output: The number of inversions of A —the number of pairs (i, j) of array indices with $i < j$ and $A[i] > A[j]$.

²Full disclosure: For plenty of important problems out there in the wild, the divide-and-conquer paradigm offers no help. Later books in this series cover additional algorithm design paradigms, such as greedy algorithms and dynamic programming, that you can resort to if the divide-and-conquer approach fails.

For example, an array A that is in sorted order has no inversions. You should convince yourself that the converse is also true: Every array that is not in sorted order has at least one inversion.

3.2.2 An Example

Consider the following array of length 6:

1	3	5	2	4	6
---	---	---	---	---	---

How many inversions does this array have? One that jumps out is the 5 and 2 (corresponding to $i = 3$ and $j = 4$). There are exactly two other out-of-order pairs: the 3 and the 2, and the 5 and the 4.

Quiz 3.1

What is the largest-possible number of inversions a 6-element array can have?

- a) 15
- b) 21
- c) 36
- d) 64

(See Section 3.2.13 for the solution and discussion.)

3.2.3 Collaborative Filtering

Why would you want to count the number of inversions in an array? One reason is to compute a numerical similarity measure that quantifies how close two ranked lists are to each other. For example, suppose I ask you and a friend to rank, from favorite to least favorite, ten movies that you have both seen. Are your tastes “similar” or “different”? One way to answer this question quantitatively is through the following 10-element array A : $A[1]$ contains your friend’s ranking of your favorite movie in their list, $A[2]$ your friend’s personal ranking of your second-favorite movie, $\dots$, and $A[10]$ your friend’s personal

ranking of your least favorite movie. So if your favorite movie is *Star Wars* but your friend has it only fifth in their list, then $A[1] = 5$. If your rankings are identical, this array will be sorted and have no inversions. The more inversions the array has, the more pairs of movies on which you disagree about their relative merits, and the more different your preferences.

One reason you might want a similarity measure between rankings is to do *collaborative filtering*, a technique for generating recommendations. How do Web sites come up with suggestions for products, movies, songs, news stories, and so on? In collaborative filtering, the idea is to identify other users who have similar preferences, and to recommend to you things that have been popular with them. Thus collaborative filtering algorithms require a formal notion of “similarity” between users, and the problem of computing inversions captures some of the essence of this problem.

3.2.4 Exhaustive Search

How quickly can we compute the number of inversions in an array? If we’re feeling unimaginative, there’s always exhaustive search.

Exhaustive Search for Counting Inversions

Input: array A of n distinct integers.

Output: the number of inversions of A .

```
numInv := 0
for i := 1 to n - 1 do
  for j := i + 1 to n do
    if  $A[i] > A[j]$  then
      numInv := numInv + 1
return numInv
```

This is certainly a correct algorithm. What about its running time? From the solution to Quiz 3.1, we know that the number of loop iterations grows quadratically with the length n of the input array. Because the algorithm performs a constant number of operations in

each loop iteration, its asymptotic running time is $\Theta(n^2)$. Remember the mantra of a seasoned algorithm designer: *can we do better?*

3.2.5 A Divide-and-Conquer Approach

The answer is yes, and the solution will be a divide-and-conquer algorithm that runs in $O(n \log n)$ time, a big improvement over the exhaustive search algorithm. The “divide” step will be exactly as in the **MergeSort** algorithm, with one recursive call for the left half of the array and one for the right half. To understand the residual work that needs to be done outside the two recursive calls, let’s classify the inversions (i, j) of an array A of length n into one of three types:

1. *left inversion*: an inversion with i, j both in the first half of the array (i.e., $i, j \leq \frac{n}{2}$);³
2. *right inversion*: an inversion with i, j both in the second half of the array (i.e., $i, j > \frac{n}{2}$);
3. *split inversion*: an inversion with i in the left half and j in the right half (i.e., $i \leq \frac{n}{2} < j$).

For example, in the six-element array in Section 3.2.2, all three of the inversions are split inversions.

The first recursive call, on the first half of the input array, recursively counts all the left inversions (and nothing else). Similarly, the second recursive call counts all the right inversions. The remaining task is to count the inversions not counted by either recursive call—the split inversions. This is the “combine” step of the algorithm, and we will need to implement a special linear-time subroutine for it, analogous to the **Merge** subroutine in the **MergeSort** algorithm.

3.2.6 High-Level Algorithm

Our divide-and-conquer approach translates to the following pseudocode; the subroutine **CountSplitInv** is, as of now, unimplemented.

³The abbreviation “i.e.” stands for *id est*, and means “that is.”

CountInv**Input:** array A of n distinct integers.**Output:** the number of inversions of A .

```

if  $n = 0$  or  $n = 1$  then                                // base cases
    return 0
else
     $leftInv := \text{CountInv}(\text{first half of } A)$ 
     $rightInv := \text{CountInv}(\text{second half of } A)$ 
     $splitInv := \text{CountSplitInv}(A)$ 
    return  $leftInv + rightInv + splitInv$ 

```

The first and second recursive calls count the number of left and right inversions. Provided the subroutine **CountSplitInv** correctly computes the number of split inversions, **CountInv** correctly computes the total number of inversions.

3.2.7 Key Idea: Piggyback on MergeSort

Counting the number of split inversions of an array in linear time is an ambitious goal. There can be a lot of split inversions: If A consists of the numbers $\frac{n}{2} + 1, \dots, n$ in order, followed by the numbers $1, 2, \dots, \frac{n}{2}$ in order, there are $n^2/4$ split inversions. How could we ever count a quadratic number of things with only a linear amount of work?

The inspired idea is to design our recursive inversion-counting algorithm so that it piggybacks on the **MergeSort** algorithm. This involves demanding more from our recursive calls, in service of making it easier to count the number of split inversions. Each recursive call will be responsible not only for counting the number of inversions in the array that it is given, but also for returning a sorted version of the array. We already know (from Theorem 1.2) that sorting is a for-free primitive (see page 31), running in $O(n \log n)$ time, so if we're shooting for a running time bound of $O(n \log n)$, there's no reason not to sort. And we'll see that the task of merging two sorted subarrays is tailor-made for uncovering all the split inversions of an array.

Here is the revised version of the pseudocode in Section 3.2.6, which counts inversions while also sorting the input array.

Sort-and-CountInv**Input:** array A of n distinct integers.**Output:** sorted array B with the same integers, and the number of inversions of A .

```

if  $n = 0$  or  $n = 1$  then                                // base cases
    return  $(A, 0)$ 
else
     $(C, leftInv) := \text{Sort-and-CountInv}(\text{first half of } A)$ 
     $(D, rightInv) :=$ 
         $\text{Sort-and-CountInv}(\text{second half of } A)$ 
     $(B, splitInv) := \text{Merge-and-CountSplitInv}(C, D)$ 
    return  $(B, leftInv + rightInv + splitInv)$ 

```

We still need to implement the **Merge-and-CountSplitInv** subroutine. We know how to merge two sorted lists in linear time, but how can we piggyback on this work to also count the number of split inversions?

3.2.8 Merge Revisited

To see why merging sorted subarrays naturally uncovers split inversions, let's revisit the pseudocode for the **Merge** subroutine.

Merge**Input:** sorted arrays C and D (length $n/2$ each).**Output:** sorted array B (length n).**Simplifying assumption:** n is even.

```

 $i := 1, j := 1$ 
for  $k := 1$  to  $n$  do
    if  $C[i] < D[j]$  then
         $B[k] := C[i], i := i + 1$ 
    else                                //  $D[j] < C[i]$ 
         $B[k] := D[j], j := j + 1$ 

```

To review, the **Merge** subroutine walks one index down each of the sorted subarrays in parallel (i for C and j for D), populating the

output array (B) from left to right in sorted order (using the index k). At each iteration of the loop, the subroutine identifies the smallest element that it hasn't yet copied over to B . Because C and D are sorted, and all the elements before $C[i]$ and $D[j]$ have already been copied over to B , the only two candidates for the smallest-remaining element are $C[i]$ and $D[j]$. The subroutine determines which of the two is smaller and then copies it over to the next position of the output array.

What does the **Merge** subroutine have to do with counting the number of split inversions? Let's start with the special case of an array A that has no split inversions at all—every inversion of A is either a left or a right inversion.

Quiz 3.2

Suppose the input array A has no split inversions. What is the relationship between the sorted subarrays C and D ?

- a) C has the smallest element of A , D the second-smallest, C the third-smallest, and so on.
- b) All the elements of C are less than all the elements of D .
- c) All the elements of C are greater than all the elements of D .
- d) There is not enough information to answer this question.

(See Section 3.2.13 for the solution and discussion.)

After solving Quiz 3.2, you can see that **Merge** has a particularly boring execution on an array with no split inversions. Because every element of C is smaller than every element of D , the smallest remaining element is always in C (until no elements of C remain). Thus the **Merge** subroutine simply concatenates C and D —it will first copy over all of C , and then all of D . This suggests that, perhaps, split inversions have something to do with the number of elements remaining in C when an element of D is copied over to the output array.

3.2.9 Merge and Split Inversions

To build our intuition further, let's think about running the **MergeSort** algorithm on the six-element array $A = \{1, 3, 5, 2, 4, 6\}$ from Section 3.2.2; see also Figure 3.1. The left and right halves of this array are already sorted, so there are no left inversions or right inversions, and the two recursive calls return 0. In the first iteration of the **Merge** subroutine, the first element of C (the 1) is copied over to B . This tells us nothing about any split inversions, and indeed there are no split inversions that involve this element. In the second iteration, however, the 2 is copied over to the output array, even though C still contains the elements 3 and 5. This exposes two of the split inversions of A —the two such inversions that involve the 2. In the third iteration, the 3 is copied over from C and there are no further split inversions that involve this element. When the 4 is copied over from D , the array C still contains a 5, and this copy exposes the third and final split inversion of A (involving the 5 and the 4).

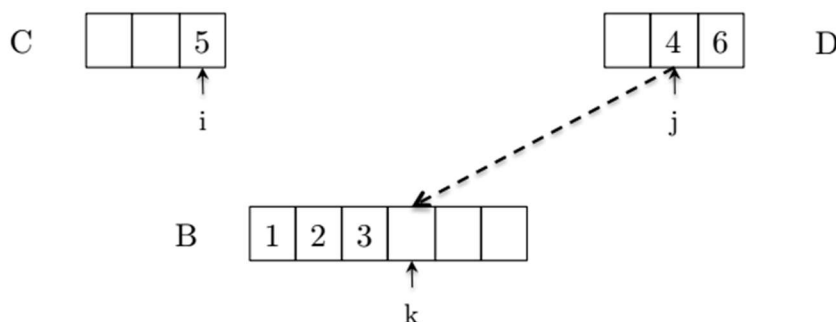


Figure 3.1: The fourth iteration of the **Merge** subroutine with the sorted input subarrays $\{1, 3, 5\}$ and $\{2, 4, 6\}$. Copying the 4 from D , with the 5 still in C , exposes the split inversion involving these two elements.

The following lemma states that the pattern in the example above holds in general: The number of split inversions that involve an element y of the second subarray D is precisely the number of elements remaining in C in the iteration of the **Merge** subroutine in which y is copied to the output array.

Lemma 3.1 (Identifying Split Inversions in Merge) *Let A be an array, and C and D sorted versions of the first and second halves*

of A . An element x from the first half of A and an element y from the second half of A form a split inversion if and only if, in the **Merge** subroutine with inputs C and D , y is copied to the output array before x .

Proof: Because the output array is populated from left to right in sorted order, the smaller of x or y is copied over first. Because x is in the first half of A and y in the second half, x and y form a split inversion if and only if $x > y$, and this is true if and only if y is copied over to the output array before x . $\mathcal{QED}$

3.2.10 Merge-and-CountSplitInv

With the insight provided by Lemma 3.1, we can extend the implementation of **Merge** to an implementation of **Merge-and-CountSplitInv**. We maintain a running count of the split inversions, and every time an element is copied over from the second subarray D to the output array B , we increment the running count by the number of elements remaining in the first subarray C .⁴

Merge-and-CountSplitInv

Input: sorted arrays C and D (length $n/2$ each).
Output: sorted array B (length n) and the number of split inversions.
Simplifying assumption: n is even.

```

 $i := 1, j := 1, splitInv := 0$ 
for  $k := 1$  to  $n$  do
  if  $C[i] < D[j]$  then
     $B[k] := C[i], i := i + 1$ 
  else                                     //  $D[j] < C[i]$ 
     $B[k] := D[j], j := j + 1$ 
     $splitInv := splitInv + \underbrace{(\frac{n}{2} - i + 1)}_{\text{\# left in } C}$ 
return  $(B, splitInv)$ 

```

⁴As with **Merge**, a full implementation should also keep track of when the traversal of C or D falls off the end of the array.

3.2.11 Correctness

Correctness of **Merge-and-CountSplitInv** follows from Lemma 3.1. Every split inversion involves exactly one element y from the second subarray, and this inversion is counted exactly once, when y is copied over to the output array. Correctness of the entire **Sort-and-CountInv** algorithm (Section 3.2.7) follows: the first recursive call correctly computes the number of left inversions, the second recursive call the number of right inversions, and **Merge-and-CountSplitInv** the remaining (split) inversions.

3.2.12 Running Time

We can also analyze the running time of the **Sort-and-CountInv** algorithm by piggybacking on the analysis we already did for the **MergeSort** algorithm. First consider the running time of a single invocation of the **Merge-and-CountSplitInv** subroutine, given two subarrays of length $\ell/2$ each. Like the **Merge** subroutine, it performs a constant number of operations per loop iteration, plus a constant number of additional operations, for a running time of $O(\ell)$.

Looking back at our running time analysis of the **MergeSort** algorithm in Section 1.5, we can see that there were three important properties of the algorithm that led to the running time bound of $O(n \log n)$. First, each invocation of the algorithm makes two recursive calls. Second, the length of the input is divided in half with each level of recursion. Third, the amount of work done in a recursive call, not counting work done by later recursive calls, is linear in the size of that call's input. Because the **Sort-and-CountInv** algorithm shares these three properties, the analysis in Section 1.5 carries over, again giving a running time bound of $O(n \log n)$.

Theorem 3.2 (Counting Inversions) *For every input array A of length $n \geq 1$, the **Sort-and-CountInv** algorithm correctly computes the number of inversions of A and runs in $O(n \log n)$ time.*

3.2.13 Solutions to Quizzes 3.1–3.2

Solution to Quiz 3.1

Correct answer: (a). The correct answer to this question is 15. The maximum-possible number of inversions is at most the number

of ways of choosing $i, j \in \{1, 2, \dots, 6\}$ with $i < j$. The latter quantity is denoted $\binom{6}{2}$, for “6 choose 2.” In general, $\binom{n}{2} = \frac{n(n-1)}{2}$, and so $\binom{6}{2} = 15$.⁵ In a six-element array $[6, 5, \dots, 1]$ sorted in reverse order, every pair of elements is out of order, and so this array achieves 15 inversions.

Solution to Quiz 3.2

Correct answer: (b). In an array with no split inversions, everything in the first half is less than everything in the second half. If there *was* an element $A[i]$ in the first half (with $i \in \{1, 2, \dots, \frac{n}{2}\}$) greater than an element $A[j]$ in the second half (with $j \in \{\frac{n}{2} + 1, \frac{n}{2} + 2, \dots, n\}$), (i, j) would constitute a split inversion.

3.3 Strassen’s Matrix Multiplication Algorithm

This section applies the divide-and-conquer algorithm design paradigm to the problem of multiplying matrices, culminating in Strassen’s amazing subcubic-time matrix multiplication algorithm. This algorithm is a canonical example of the magic and power of clever algorithm design—of how algorithmic ingenuity can improve over straightforward solutions, even for extremely fundamental problems.

3.3.1 Matrix Multiplication

Suppose $\mathbf{X}$ and $\mathbf{Y}$ are $n \times n$ matrices of integers— n^2 entries in each. In the product $\mathbf{Z} = \mathbf{X} \cdot \mathbf{Y}$, the entry z_{ij} in the i th row and j th column of $\mathbf{Z}$ is defined as the dot product of the i th row of $\mathbf{X}$ and the j th column of $\mathbf{Y}$ (Figure 3.2).⁶ That is,

$$z_{ij} = \sum_{k=1}^n x_{ik} y_{kj}. \quad (3.1)$$

⁵There are $n(n-1)$ ways to choose (i, j) so that $i \neq j$ (n choices for i , then $n-1$ for j). By symmetry, $i < j$ in exactly half of these.

⁶To compute the *dot product* of two length- n vectors $\mathbf{a} = (a_1, \dots, a_n)$ and $\mathbf{b} = (b_1, \dots, b_n)$, add up the results of multiplying componentwise. Or in symbols, $\mathbf{a} \cdot \mathbf{b} = \sum_{k=1}^n a_k b_k$.